Taller: Peticiones en Django

Endpoints con vistas basadas en clase (GET, POST, PUT y DELETE)

Docente: Jose Javier Torres Vanstrahlen

Curso: Desarrollo de aplicaciones Web I - 4E

Estudiante: Emily Esther Novoa Mendoza

Fecha: 29 de octubre de 2026

1. ¿Qué es un endpoint?

Un endpoint es una URL de la aplicación que recibe peticiones HTTP y devuelve una respuesta (en este taller, en formato JSON). En Django nativo, sin librerías externas, se construye con dos archivos: **views.py**, que contiene la lógica, y **urls.py**, que conecta una URL con esa vista.

2. Verbos HTTP y operaciones CRUD

Verbo HTTP	Operación CRUD	Caso de uso	Método en la vista
GET	Read (leer)	Consultar la lista de proyectos o uno solo	get()
POST	Create (crear)	Registrar un proyecto nuevo	post()
PUT	Update (actualizar)	Modificar un proyecto existente	put()
DELETE	Delete (eliminar)	Borrar un proyecto	delete()

GET solo consulta y no modifica datos. POST envía los datos en el cuerpo (body) de la petición. PUT y DELETE necesitan identificar el registro, por eso la URL lleva el **pk** (por ejemplo, /projects/3/). PUT reemplaza el recurso, mientras que PATCH lo modifica parcialmente.

3. Vista basada en clase

Se crea una clase que hereda de **django.views.View**. Cuando llega una petición, el método **dispatch()** revisa el verbo HTTP y llama al método de la clase con el mismo nombre (get, post, put o delete). Si el verbo no está definido en la clase, Django responde con el error 405 (método no permitido). En urls.py la clase se registra con **as_view()**, que la convierte en una función que Django puede usar como vista.

4. Módulos, funciones y decoradores usados

Elemento	Módulo	Descripción
View	django.views	Clase base de las vistas basadas en clase. Envía cada petición al método según el verbo HTTP.
as_view()	método de View	Convierte la clase en una vista invocable para usarla en urls.py.
dispatch()	método de View	Decide qué método ejecutar (get, post, put, delete) según el verbo de la petición.
http_method_not_allowed()	método de View	Devuelve el error 405 cuando el verbo no está implementado.
HttpRequest (request)	django.http	Objeto con los datos de la petición: request.method, request.body, etc.
JsonResponse	django.http	Devuelve la respuesta en formato JSON. Con safe=False permite responder una lista.
json.loads()	librería estándar json	Convierte el cuerpo de la petición (texto JSON) en un diccionario de Python.

Elemento	Módulo	Descripción
path()	django.urls	Define una ruta. Con <int:pk> captura un número entero de la URL y lo pasa a la vista.
include()	django.urls	Conecta el urls.py de una aplicación con el urls.py principal del proyecto.
@method_decorator	django.utils.decorators	Permite aplicar un decorador pensado para funciones a un método de una clase.
@csrf_exempt	django.views.decorators.csrf	Desactiva la verificación del token CSRF, necesaria para probar POST, PUT y DELETE desde herramientas externas.
model_to_dict()	django.forms.models	Convierte un objeto del modelo en un diccionario (no incluye el id, por eso se agrega aparte).
objects.all()	ORM de Django	Obtiene todos los registros del modelo.
objects.get()	ORM de Django	Obtiene un registro por su clave. Lanza Model.DoesNotExist si no existe.
objects.create()	ORM de Django	Crea y guarda un registro nuevo.
save()	ORM de Django	Guarda los cambios de un objeto existente.
delete()	ORM de Django	Elimina el registro de la base de datos.
IntegrityError	django.db	Error que aparece cuando los datos violan una restricción de la base de datos.

5. Archivos modificados

views.py (app gestion): clase ProjectView con los cuatro métodos.
urls.py (app gestion): dos rutas, projects/ y projects/<int:pk>/.
urls.py (proyecto my_feel): se agregó path("api/", include("gestion.urls")).

6. Evidencias de funcionamiento

Pruebas realizadas con el servidor de desarrollo (python manage.py runserver). GET se probó desde el navegador; POST, PUT y DELETE se probaron con PowerShell (Invoke-RestMethod).

GET: leer

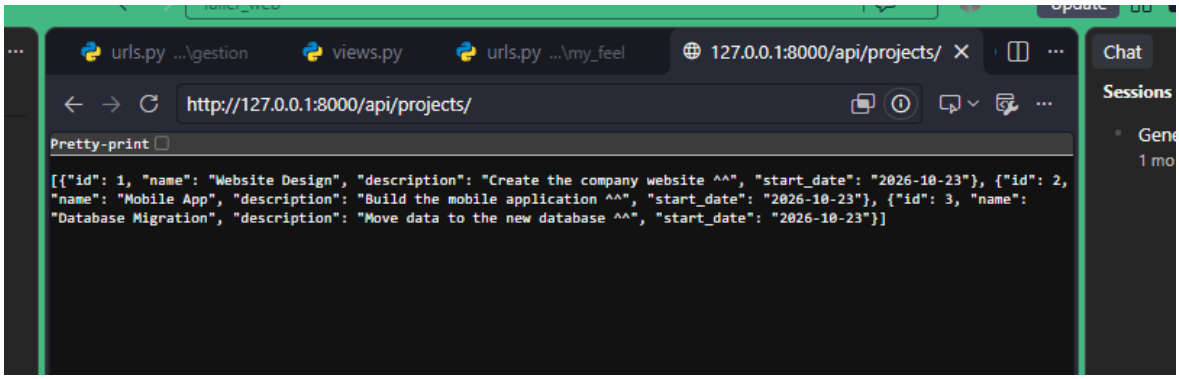


Figura 1. GET /api/projects/ devuelve la lista con los tres proyectos (Read).



Figura 2. GET /api/projects/1/ devuelve un solo proyecto usando el pk en la URL.

POST y PUT: crear y actualizar

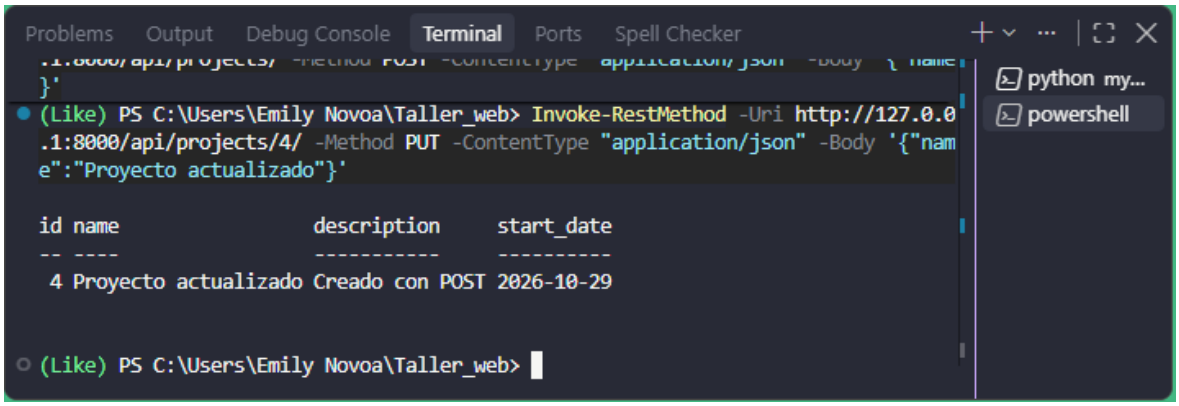


Figura 3. PowerShell: en la parte superior se ve el comando POST con el que se creó el proyecto 4 (Create) y, debajo, el PUT a /api/projects/4/ que cambió su nombre a "Proyecto actualizado" (Update).

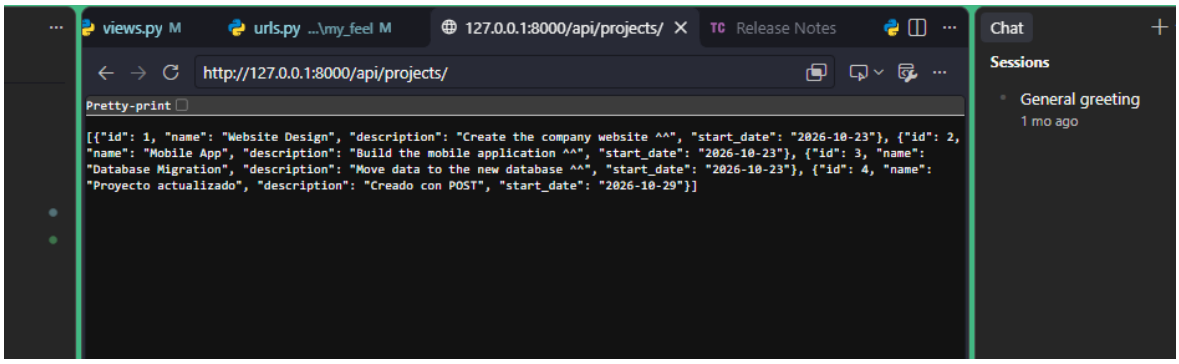


Figura 4. GET /api/projects/ después del POST y el PUT: aparece el proyecto 4 con el nombre actualizado.

DELETE: eliminar

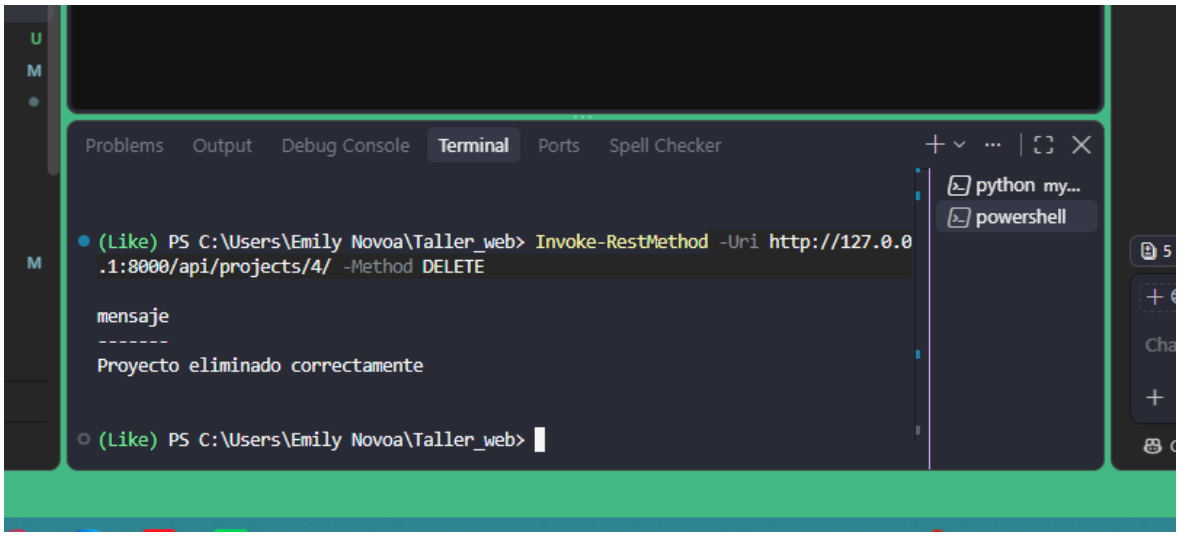


Figura 5. DELETE a /api/projects/4/ responde "Proyecto eliminado correctamente" (Delete).

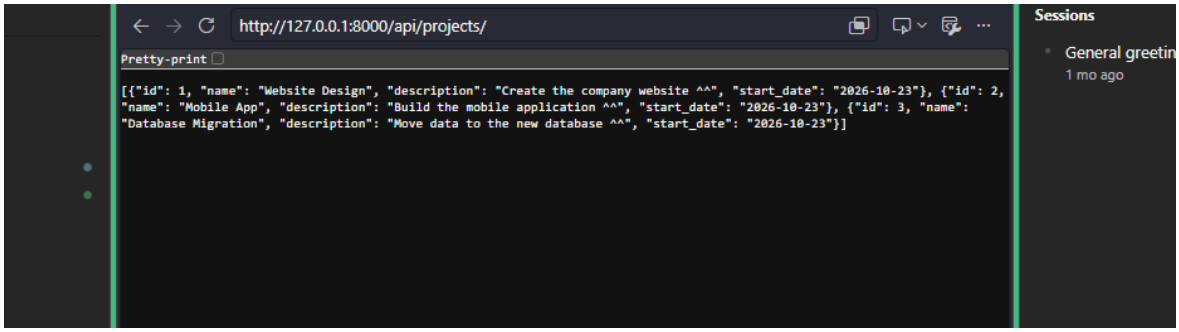


Figura 6. GET /api/projects/ después del DELETE: quedan solo los tres proyectos originales.

7. Fuentes consultadas

- Django Software Foundation. Base views (View, dispatch, as_view): https://docs.djangoproject.com/en/stable/ref/class-based-views/base/
- Django Software Foundation. Introduction to class-based views: https://docs.djangoproject.com/en/stable/topics/class-based-views/intro/

• Django Software Foundation. Request and response objects (HttpRequest, JsonResponse): https://docs.djangoproject.com/en/stable/ref/request-response/

• Django Software Foundation. URL dispatcher (path, include): https://docs.djangoproject.com/en/stable/topics/http/urls/

• Django Software Foundation. Cross Site Request Forgery protection (csrf_exempt): https://docs.djangoproject.com/en/stable/howto/csrf/

• Django Software Foundation. Making queries (all, get, create, save, delete): https://docs.djangoproject.com/en/stable/topics/db/queries/

• Mozilla Developer Network. Métodos de petición HTTP: https://developer.mozilla.org/es/docs/Web/HTTP/Methods